

Aula 2: Banco de dados, ActiveRecord e o model `User`

Você sai daqui com: um `User` com validações e senha hasheada, testado. **Gabarito:** `cd ~/capacitacao-gabarito && git checkout aula-02`

De onde viemos, e para onde vamos hoje

Na Aula 1 você subiu uma API que responde `GET /api/v1/status`. Ela funciona, e não guarda nada: desligou o servidor, acabou.

Hoje ela ganha memória. Você vai criar as tabelas, escrever o model `User`, guardar senha do jeito certo (que não é guardar senha) e ligar duas tabelas uma na outra. No fim, um usuário criado no console continua lá depois de você reiniciar tudo.

É a aula em que você mais escreve código nas quatro, e é a base da Aula 3, em que esse `User` vira cadastro, login e recuperação de senha de verdade.

Travou? A tabela **Se der errado** no fim de cada prática cobre os erros que realmente acontecem. Consulte antes de chamar, e chame se não resolver em dez minutos.

1. Por que Postgres, e não SQLite

SQLite é um arquivo. Funciona muito bem para um app de celular ou um script. Para um servidor com várias requisições ao mesmo tempo, ele trava: só uma escrita por vez no banco inteiro.

Postgres é um servidor: aguenta concorrência, tem tipos ricos (`jsonb`, arrays, intervalos de tempo), índices parciais e transações de verdade. É o que o `seem-backend` usa em produção, então é o que usamos aqui: **desenvolver no mesmo banco da produção evita a categoria inteira de bug que só aparece no deploy.**

Transação

Um bloco de operações que acontece **inteiro ou não acontece**. Deu erro no meio, o banco desfaz tudo: isso se chama *rollback*.

O exemplo clássico é transferir dinheiro: debitar de um e creditar no outro têm que ser a mesma operação. Debitar sozinho é dinheiro que sumiu.

```
User.transaction do
  user.save!
  user.invalidate_sessions!
  user.clear_password_reset_code!
end
```

Repare no `!`: dentro de uma transação você **quer** que estoure. `save` devolve `false` em silêncio e a transação seguiria feliz, gravando metade.

Na Aula 3, trocar a senha vai precisar exatamente disso.

2. ActiveRecord

É o ORM do Rails: cada classe é uma tabela, cada objeto é uma linha.

Django ORM	ActiveRecord
<code>User.objects.filter(role="admin")</code>	<code>User.where(role: "admin")</code>
<code>User.objects.get(id=1)</code>	<code>User.find(1)</code>
<code>User.objects.all().order_by("-created_at")</code>	<code>User.order(created_at: :desc)</code>
<code>User.objects.count()</code>	<code>User.count</code>
<code>u.save()</code>	<code>u.save</code>

A diferença de filosofia: no Django você declara os campos na classe. **No Rails a classe não declara nada**: ela lê as colunas do banco em tempo de execução. A fonte da verdade é a migration.

```
class User < ApplicationRecord
end
```

Isso já tem `name`, `email`, `id`, `created_at`: tudo que existir na tabela `users`.

ActiveSupport: os métodos que o Rails inventou

O Rails adiciona métodos às classes do próprio Ruby. Isso é a gem `activesupport`, não Ruby puro, fora do Rails esses métodos somem.

```
15.minutes
1.day.ago
2.weeks.from_now
Time.current      # respeita o fuso configurado na app; Time.now não
```

E o par que você vai usar mais que qualquer outro:

```
nil.blank?      # true
"".blank?       # true
"  ".blank?     # true  ← só espaço também conta como vazio
[].blank?       # true
0.blank?        # false  ← zero NÃO é vazio

"oi".present?   # true  ← present? é o contrário de blank?
```

Pegadinha para quem vem de Python: em Ruby puro, só `nil` e `false` são falsos. `if 0` executa. `if ""` executa. É justamente por isso que `blank?` existe.

3. Migrations

Uma migration é uma **mudança no banco, versionada em código**. Ela roda uma vez, em ordem, em todas as máquinas: a sua, a do colega e o servidor.

```
bin/rails generate migration CreateUsers
```

Isso cria `db/migrate/20260803142949_create_users.rb`. O número é a data e hora: é ele que define a ordem.

```
class CreateUsers < ActiveRecord::Migration[8.1]
  def change
    create_table :users do |t|
      t.string :name, null: false
      t.string :email, null: false
      t.string :password_digest, null: false
      t.string :course, null: false
      t.string :matricula, null: false
      t.datetime :terms_accepted_at, null: false

      t.timestamps
    end

    add_index :users, :email, unique: true
    add_index :users, :matricula, unique: true
  end
end
```

```
bin/rails db:migrate
```

Repare em duas coisas:

- `t.timestamps` cria `created_at` e `updated_at`, e o Rails mantém as duas sozinho.
- `add_index ... unique: true` é uma restrição no banco, não só validação no model. Isso importa: duas requisições simultâneas passam pela validação juntas (as duas consultam e as duas não acham ninguém), mas só uma vence o índice único. **Validação de model é para dar mensagem bonita; índice é para garantir.**

`schema.rb`

Depois de migrar, o Rails reescreve `db/schema.rb`: o retrato atual do banco. É ele que `db:prepare` usa para criar o banco de teste, e por isso **vai versionado**. Você nunca edita esse arquivo à mão.

Migrations são incrementais

Neste projeto a tabela `users` nasceu em três migrations, porque as funcionalidades chegaram em momentos diferentes:

```
20260803142949_create_users.rb          # o básico
20260803142950_add_email_verification_to_users.rb # confirmação de e-mail
20260803142951_add_password_reset_to_users.rb   # recuperação de senha
```

É assim que acontece na vida real. Nunca edite uma migration que já rodou em produção: crie outra.

4. Senha: o que nunca fazer

Nunca guarde senha em texto. Se o banco vazar, e bancos vazam, você entregou a senha de todos. E como as pessoas repetem senha, você entregou o e-mail e o banco delas junto.

Hash não é criptografia

Criptografia tem volta: basta ter a chave. **Hash não tem volta**, é um caminho só. Você não guarda a senha; guarda o resultado de passar a senha pela função. Na hora do login, passa de novo e compara os resultados.

Por que bcrypt e não SHA-256

SHA-256 é rápido: **e isso é o problema**. Uma GPU testa bilhões de SHA-256 por segundo. bcrypt foi feito para ser **lento de propósito** e tem um fator de custo ajustável: quando o hardware melhora, você aumenta o custo.

bcrypt também gera um **salt** aleatório por senha. Sem salt, duas pessoas com a mesma senha teriam o mesmo hash, e uma tabela pronta (*rainbow table*) quebraria as duas de uma vez.

```
$2a$12$R9h/CIPz0gi.URNXX3kh20...
|   |   └ salt + hash
|   └ fator de custo (12 = 2^12 iterações)
└ versão do algoritmo
```

has_secure_password

```
# Gemfile
gem "bcrypt", "~> 3.1.7"
```

```
class User < ApplicationRecord
  has_secure_password validations: false
end
```

Isso dá ao model:

- `user.password = "..."`: recebe a senha em texto e grava o digest em `password_digest`
- `user.authenticate("...")`: devolve o usuário se bater, `false` se não
- `user.password_confirmation`: a confirmação, se você usar

A senha em texto **nunca toca o banco**. Ela vive na memória durante a requisição e some.

`validations: false` desliga as validações padrão porque a nossa política de senha é mais exigente. Ela mora em `PasswordPolicyValidator`.

5. Validações

```
validates :name, presence: true, length: { maximum: 120 }
validates :email, presence: true,
              uniqueness: { case_sensitive: false },
              format: { with: URI::MailTo::EMAIL_REGEXP }
validates :course, presence: true
validates :matricula, presence: true, uniqueness: true, format: { with: /\A\d{7}\z/ }
validates :terms_accepted_at, presence: true, on: :create
validate :password_meets_policy, if: -> { password.present? }
```

A última linha chama um método **seu**, que você escreve no fim da classe, depois de um `private`. É ele que liga o validador da prática anterior ao ActiveRecord:

```
private

def password_meets_policy
  PasswordPolicyValidator.validate(password).each do |message|
    errors.add(:password, message)
  end
end
```

`errors.add` é como se acrescenta erro na mão. Cada mensagem que o validador devolveu vira um erro do campo `password`, e é por isso que `u.errors.full_messages` mostra todas de uma vez.

- `validates` (plural) usa um validador pronto. `validate` (singular) chama um método seu.
- `on: :create` só valida na criação: os termos são aceitos uma vez.
- `if:` recebe um lambda. Só valida a senha quando ela foi informada (na edição de perfil ela não é).

No console:

```
u = User.new(name: "Teste")
u.valid?          # false
u.errors.full_messages
```

Regra de ouro: normalize antes de validar

`Ana@UFOP.br` e `ana@ufop.br` são o mesmo e-mail. `20.112-34` e `2011234` são a mesma matrícula.

```
def self.normalize_email(email)
  email.to_s.strip.downcase
end

def self.normalize_matricula(matricula)
  matricula.to_s.gsub(/\D/, "")
end
```

Sem isso o índice único não serve para nada: o banco acha que são valores diferentes.

Repare que são **métodos de classe**, e não callbacks: eles não rodam sozinhos ao salvar. Quem os chama é quem recebe o dado de fora, e nesta aula é o `authenticate_by_email` (seção 11). Na Aula 3, o service de cadastro chama os três antes de criar o usuário.

É uma escolha, e tem motivo: normalizar num callback esconde a transformação de quem lê o service, e faz o mesmo dado se comportar diferente conforme o caminho que ele tomou para chegar ali.

6. Associações

Uma tabela nunca basta. Todo sistema real tem tabelas que se referem umas às outras: usuário tem muitos pedidos; palestra acontece numa sala; sala tem muitas palestras.

Vamos criar a segunda tabela do projeto: `login_events`, o histórico de acessos da conta. É o que o seu banco usa para mandar "novo acesso detectado" por e-mail.

Quem guarda a chave

A relação um-para-muitos mora numa coluna só: a **chave estrangeira**. `login_events.user_id` aponta para `users.id`.

- quem **carrega** a coluna usa `belongs_to`;
- quem é **apontado** usa `has_many`.

Regra prática: a chave fica sempre no lado "muitos".

A migration

```
create_table :login_events do |t|
  t.references :user, null: false, foreign_key: true
  t.string :client, null: false
  t.string :ip_address
  t.datetime :occurred_at, null: false
  t.timestamps
end

add_index :login_events, [ :user_id, :occurred_at ]
```

- `t.references :user` cria a coluna `user_id`, o índice **e** a chave estrangeira.
- `foreign_key: true` faz o **banco** recusar uma linha órfã: não é só validação de model.
- O índice composto tem `user_id` primeiro porque a consulta é sempre "os últimos logins **deste** usuário". Num índice composto, **a ordem das colunas importa**: primeiro o que filtra.

Os dois lados

```
class User < ApplicationRecord
  has_many :login_events, dependent: :delete_all
end

class LoginEvent < ApplicationRecord
  belongs_to :user

  scope :recentes, -> { order(occurred_at: :desc) }
end
```

- `dependent: :delete_all`: apagar o usuário apaga o histórico junto. Sem isso sobram linhas apontando para um `id` que não existe mais.
- `belongs_to` **já exige presença** desde o Rails 5: `LoginEvent` sem `user` é inválido.
- `scope` é uma consulta com nome, e ela encadeia.

Usando

```
# Criar PELA associação já preenche o user_id – não precisa passar:
user.login_events.create!(client: "mobile", occurred_at: Time.current)

user.login_events.count
user.login_events.recentes.limit(5)
evento.user.name # navega para o outro lado
```

A armadilha N+1

```
users.each { |u| puts u.login_events.count }
```

Com 100 usuários isso são **101 consultas**: uma para buscar os usuários e cem para buscar o histórico de cada um. É o problema de performance número 1 de aplicação Rails.

A correção é uma palavra:

```
User.includes(:login_events).each { |u| puts u.login_events.count }
```

Duas consultas, sempre. O `seem-backend` usa a gem **Bullet**, que estoura um erro em desenvolvimento quando você escreve um N+1 sem perceber.

7. Concerns: o mixin da Aula 1, na prática

O `User` vai ganhar confirmação de e-mail e recuperação de senha. São dois assuntos que não conversam entre si. Jogar os dois no `user.rb` produz um arquivo de 800 linhas que ninguém abre com vontade.

Cada assunto vira um módulo em `app/models/concerns/`:


```
# app/models/concerns/email_verifiable.rb
module EmailVerifiable
  extend ActiveSupport::Concern

  CODE_TTL = 15.minutes
  RESEND_COOLDOWN = 1.minute

  included do
    scope :email_verified, -> { where.not(email_verified_at: nil) }
  end

  def email_verified?
    email_verified_at.present?
  end

  def issue_email_verification_code!
    code = format("%06d", SecureRandom.random_number(1_000_000))

    update!(
      email_verification_code_digest: BCrypt::Password.create(code),
      email_verification_expires_at: CODE_TTL.from_now,
      email_verification_sent_at: Time.current
    )

    code
  end
end
```

```
class User < ApplicationRecord
  include EmailVerifiable
  include PasswordResetable
end
```

- `extend ActiveSupport::Concern` habilita o bloco `included do ... end`, onde vão coisas que só fazem sentido na classe (scopes, validações, associações).
- O código completo dos dois concerns está em `app/models/concerns/`.

Três decisões dentro do concern, e o porquê de cada uma

- 1. O código de 6 dígitos também vira digest bcrypt.** Ele é uma senha temporária. Quem lê o banco não pode confirmar a conta de ninguém.
- 2. O código em texto só existe no retorno do método.** `issue_email_verification_code!` devolve o código para o service mandar por e-mail, e depois ele some. Não fica em coluna, nem em log.
- 3. `email_verified_at` é data, não booleano.** Um booleano responde "sim". Uma data responde "sim, em 12 de agosto às 14h", e isso você vai querer saber quando alguém abrir um chamado.

8. O console

```
bin/rails console
```

```
u = User.new(
  name: "Ana Souza",
  email: "ana@aluno.ufop.edu.br",
  password: "Automic@2026",
  course: "Engenharia de Controle e Automação",
  matricula: "2011234",
  terms_accepted_at: Time.current
)

u.valid?
u.save
u.password_digest      # o hash, não a senha
u.authenticate("Automic@2026") # devolve o user
u.authenticate("errada")      # false

User.count
User.where("email LIKE ?", "%ufop%")
User.find_by(email: "ana@aluno.ufop.edu.br")

codigo = u.issue_email_verification_code!
u.verify_email_code!(codigo)
u.email_verified?
```

`bin/rails console --sandbox` desfaz tudo ao sair. Bom para experimentar sem sujar o banco.

9. Seeds e fixtures

Seeds povoam o banco de desenvolvimento:

```
bin/rails db:seed
```

Fixtures são os dados dos testes, em `test/fixtures/users.yml`. O Rails carrega antes de cada teste e limpa depois: todo teste começa do mesmo estado.

```
ana:
  name: Ana Souza
  email: ana@aluno.ufop.edu.br
  password_digest: <%= BCrypt::Password.create("Automic@2026") %>
  email_verified_at: <%= 1.day.ago.to_fs(:db) %>
```

No teste, `users(:ana)` devolve esse registro.

10. Testando o model

```
test "guarda o digest e nunca a senha em texto" do
  user = build_user
  user.save!

  assert_not_equal "Automic@2026", user.password_digest
  assert user.authenticate("Automic@2026")
  assert_not user.authenticate("outra-senha")
end
```

```
bin/rails test
bin/rails test test/models/user_test.rb          # só um arquivo
bin/rails test test/models/user_test.rb:42       # só um teste
```

O Rails vem com **Minitest**. Se você conhece `pytest`, a ideia é a mesma; a sintaxe é `assert_*`.

11. Uma sutileza de segurança: `authenticate_by_email`

```
DUMMY_PASSWORD_DIGEST = BCrypt::Password.create("automic-timing-safe-dummy").freeze

def self.authenticate_by_email(email, password)
  user = find_by(email: normalize_email(email))
  digest = user&.password_digest || DUMMY_PASSWORD_DIGEST
  autenticado = BCrypt::Password.new(digest).is_password?(password.to_s)

  user if autenticado
end
```

Por que esse `DUMMY_PASSWORD_DIGEST`?

O jeito ingênuo seria `return nil unless user`. Só que bcrypt é lento **de propósito**. Se o e-mail não existe, a resposta volta em 1ms; se existe e a senha está errada, volta em 100ms. Cronometrando as respostas, dá para descobrir quem tem conta no sistema: é um **ataque de temporização**.

Rodando o bcrypt sempre, com um digest descartável quando não há usuário, as duas respostas custam o mesmo. Vamos usar esse método na Aula 3.

As práticas da aula

Oito práticas, cada uma logo depois do bloco que a explica. Todas terminam com uma conferência: não siga em frente com ela vermelha.

#	O que	Depois de
1	A gem do bcrypt, e o banco de pé	seção 1
2	As três migrations da tabela <code>users</code>	seção 3
3	Senha no console: veja o bcrypt trabalhar	seção 4
4	O validador de senha	seção 5
5	O model <code>User</code>	seção 5
6	A segunda tabela e a associação	seção 6
7	Os dois concerns	seção 7
8	Fixtures, testes e commit	seção 10

No **seu** projeto (`~/automic_auth_api`), continuando de onde a Aula 1 parou.

Prática 1: A gem do bcrypt, e o banco de pé

Aquecimento, e garanta que o ambiente da Aula 1 continua funcionando.

```
cd ~/automic_auth_api
docker compose up -d
docker compose ps          # healthy
```

No `Gemfile`, descomente (ou acrescente) a linha:

```
gem "bcrypt", "~> 3.1.7"
```

```
bundle install
```

O que você deve ver:

```
bin/rails runner 'puts BCrypt::Password.create("teste")[0, 7]'
```

Sai algo como `$2a$12$`. Esse prefixo é o algoritmo e o custo: você vai entender os dois na seção 4.

Se der errado

Erro	Causa	Saída
<code>Could not find gem 'bcrypt'</code>	esqueceu o <code>bundle install</code>	rode-o
<code>An error occurred while installing bcrypt</code>	falta compilador ou headers	Linux: <code>sudo apt install build-essential</code> ; macOS: <code>xcode-select --install</code>
<code>uninitialized constant BCrypt</code>	a linha ficou comentada no <code>Gemfile</code>	tire o <code>#</code> do começo
<code>PG::ConnectionBad</code>	o container não subiu	<code>docker compose up -d</code> e confira o <code>ps</code>

Prática 2: As três migrations da tabela `users`

A prática mais longa do dia. Faça uma migration por vez, e migre entre elas.

```
bin/rails generate migration CreateUsers
bin/rails generate migration AddEmailVerificationToUsers
bin/rails generate migration AddPasswordResetToUsers
```

O conteúdo da primeira está na seção **3. Migrations**. As outras duas:

```
# add_email_verification_to_users.rb
add_column :users, :email_verification_code_digest, :string
add_column :users, :email_verification_expires_at, :datetime
add_column :users, :email_verification_sent_at, :datetime
add_column :users, :email_verified_at, :datetime

# add_password_reset_to_users.rb
add_column :users, :password_reset_code_digest, :string
add_column :users, :password_reset_expires_at, :datetime
add_column :users, :password_reset_sent_at, :datetime
```

```
bin/rails db:migrate
```

O que você deve ver:

```
bin/rails db:migrate:status # três linhas "up"
sed -n '/create_table "users"/,/^\s*end/p' db/schema.rb | grep '^  t\.' | grep -vc 't\.index'
```

O segundo comando conta as colunas da tabela, e tem que sair **15**: as 8 da primeira migration (seis suas mais as duas do `t.timestamps`), 4 da segunda e 3 da terceira.

Repare em duas coisas que ele mostra: o `id` não está na lista, porque o `create_table` cria essa coluna sozinho e nem a menciona; e as duas linhas `t.index` que o comando descarta são os índices únicos de `email` e `matricula`, que são restrição do banco e não coluna.

Na Aula 3 aparece uma décima sexta, o `token_version`, numa migration que vem junto com o código daquele dia. É ela que permite derrubar todas as sessões de um usuário de uma vez.

Abra o `db/schema.rb` e leia: ele é o retrato do banco **agora**, montado sozinho pelas migrations.

Se der errado

Migration que falha no meio dá uma mensagem enorme e assustadora. **Ignore a primeira linha**, ela só diz que a migration parou. A resposta está na **linha seguinte**, e costuma ser bem específica. Esse hábito sozinho economiza horas ao longo de uma carreira.

E nesta fase do projeto, `db:drop db:create db:migrate` é sempre uma saída legítima: não há dado nenhum para perder. Não tenha dó.

Erro	Causa	Saída
<code>PG::DuplicateTable: relation "users" already exists</code>	you rodou a migration duas vezes, ou criou a tabela na mão	<code>bin/rails db:drop db:create db:migrate</code> e tudo bem agora)
<code>PG::DuplicateColumn</code>	mesma coisa, com coluna	idem
<code>An error has occurred, this and all later migrations canceled</code>	uma migration falhou no meio	leia a linha seguinte: é ela que diz o erro arquivo e rode de novo
<code>ActiveRecord::IrreversibleMigration</code> ao fazer <code>rollback</code>	a migration usou <code>change</code> com algo que não sabe desfazer	troque por <code>up</code> / <code>down</code> , ou refaça com <code>db</code>
o <code>schema.rb</code> não mudou	a migration não rodou	<code>bin/rails db:migrate:status</code> : a sua es
saiu menos que 15	falta alguma coluna	compare com o gabarito: <code>git -C ~/capacitacao-gabarito show a</code>
<code>Multiple migrations have the name ...</code>	you gerou duas com o mesmo nome	apague a duplicada em <code>db/migrate/</code>

Prática 3: senha no console, vendo o bcrypt trabalhar

Sem escrever arquivo nenhum. É a prática que faz a seção 4 parar de ser teoria.

```
bin/rails console
```

Antes de rodar, aposte: a mesma senha, hashada duas vezes, dá o mesmo resultado ou resultados diferentes? Decida agora: a graça do exercício está em você errar essa aposta.

```
# a) o mesmo texto, dois digests diferentes
BCrypt::Password.create("Automic@2026")
BCrypt::Password.create("Automic@2026")
```

Rodou duas vezes e deu **diferente**? Esse é o *salt*, e é ele que impede alguém de descobrir que dois usuários têm a mesma senha.

```
# b) mas os dois conferem
digest = BCrypt::Password.create("Automic@2026")
BCrypt::Password.new(digest) == "Automic@2026"    # true
BCrypt::Password.new(digest) == "errada"          # false
```

Ao rodar o bloco abaixo pode aparecer um aviso de três linhas dizendo que o `benchmark` vai sair das gems padrão no Ruby 4.0. É recado para quem mantém biblioteca, não para você. Ignore.

```
# c) por que é lento de propósito
require "benchmark"

Benchmark.realtime { BCrypt::Password.create("Automic@2026") } # UM bcrypt
Benchmark.realtime { 1000.times { Digest::SHA256.hexdigest("Automic@2026") } } # MIL SHA-256
```

O que você deve ver: o segundo comando fez **mil** hashes de SHA-256 e mesmo assim terminou muito antes do primeiro, que fez **um** bcrypt.

É essa diferença que protege o seu banco. Quem rouba a tabela de usuários testa senha em lote: com SHA-256 ele testa bilhões por segundo, e com bcrypt, algumas dezenas. Diga em voz alta por que a lentidão aqui é uma *característica* e não um defeito.

Se der errado

Erro	Causa	Saída
<code>uninitialized constant BCrypt</code>	a gem não entrou	volte para a Prática 1
<code>uninitialized constant Digest</code>	não carregou	<code>require "digest"</code> antes
o console não abre, erro de banco	container caiu	<code>docker compose up -d</code>
você fechou o console sem querer	<code>exit</code> ou <code>Ctrl+D</code>	é só reabrir; nada foi perdido

Prática 4: O validador de senha

O primeiro arquivo Ruby seu, do zero, nesta capacitação.

Crie `app/validators/password_policy_validator.rb`:

```
class PasswordPolicyValidator
  SPECIAL_CHARACTERS = %r{[!@#$%^&*(),.?":{}|<>]}
  MIN_LENGTH = 8
  MAX_LENGTH = 16

  def self.validate(password)
    new(password).validate
  end

  def initialize(password)
    @password = password.to_s
  end

  # Devolve uma lista de mensagens. Vazia significa senha aceita.
  def validate
    errors = []
    errors << "deve ter entre #{MIN_LENGTH} e #{MAX_LENGTH} caracteres" unless length_valid?
    errors << "deve incluir pelo menos uma letra maiúscula" unless @password.match?(/[A-Z]/)
    errors << "deve incluir pelo menos uma letra minúscula" unless @password.match?(/[a-z]/)
    errors << "deve incluir pelo menos um dígito" unless @password.match?(/\d/)
    errors << "deve incluir pelo menos um caractere especial" unless @password.match?(
(SPECIAL_CHARACTERS)
    errors
  end

  private

  def length_valid?
    @password.length.between?(MIN_LENGTH, MAX_LENGTH)
  end
end
```

Repare em três coisas da Aula 1: o `self.validate` que instancia e chama, o `@password` de instância, e o `private` valendo da linha em diante.

Ele fica fora do model porque o cadastro precisa validar a senha **antes** de existir um `User` (Aula 3), e a recuperação de senha valida de novo na hora de trocar.

O que você deve ver:

```
bin/rails runner 'p PasswordPolicyValidator.validate("abc")'
# a lista de erros
bin/rails runner 'p PasswordPolicyValidator.validate("Automic@2026")'
# []
```

Avançado: `PasswordPolicyValidator.validate(nil)` também tem que devolver a lista de erros, sem estourar. Descubra qual linha do código garante isso.

Se der errado

Erro	Causa	Saída
<code>uninitialized constant PasswordPolicyValidator</code>	caminho ou nome do arquivo errado	tem que ser <code>app/validators/password_policy_validator.rb</code> Zeitwerk de novo
<code>NoMethodError: undefined method 'length' for nil</code>	você tirou o <code>.to_s</code> do <code>initialize</code>	é ele que transforma <code>nil</code> em <code>""</code>
<code>undefined method 'validate' for an instance of Class</code>	escreveu <code>def validate</code> onde queria <code>def self.validate</code>	o <code>self.</code> faz o método ser da classe
a senha boa devolve erro de caractere especial	a regex saiu com escape errado	copie a linha do <code>SPECIAL_CHARACTERS</code> i
<code>syntax error, unexpected end</code>	faltou ou sobrou um <code>end</code>	conte: <code>class</code> , <code>def</code> ×4, <code>if</code> ...

Prática 5: O model `User`

Crie `app/models/user.rb` com `has_secure_password validations: false`, as seis validações, os três normalizadores e o `authenticate_by_email` (seções 4, 5 e 11).

O que você deve ver:

```
bin/rails runner '
u = User.new(name: "Ana", email: "ana@ufop.br", password: "Automic@2026",
              course: "Automação", matricula: "2011234", terms_accepted_at: Time.current)
puts u.valid?
puts u.password_digest[0, 20]
'
```

Tem que sair `true` e um digest começando em `$2a$12$`.

Agora **quebre de propósito** e leia a mensagem:

```
bin/rails runner 'u = User.new(email: "NAO-E-EMAIL"); u.valid?; p u.errors.full_messages'
```

Se der errado

Erro	Causa	Saída
<code>NoMethodError: undefined method 'password='</code>	falta o <code>has_secure_password</code>	acrescente-o no topo da classe
<code>PG::UndefinedColumn: column "password_digest" does not exist</code>	a migration não rodou	<code>bin/rails db:migrate</code>
<code>valid?</code> devolve <code>false</code> e você não sabe por quê	as mensagens estão no objeto	<code>p u.errors.full_messages</code> sempre
<code>valid?</code> devolve <code>true</code> com e-mail inválido	falta a validação de formato	releia a seção 5
o e-mail salvou com maiúscula	os normalizadores não rodam sozinhos, é preciso chamá-los	<code>User.normalize_email(...)</code> de criar, como o <code>authenticate_by_email</code> faz
<code>ArgumentError: wrong number of arguments</code>	passou posicional onde é nomeado	<code>User.new(name: ..., email: ...)</code>

Prática 6: A segunda tabela e a associação

É aqui que o banco deixa de ser uma tabela e vira um *modelo de dados*.

```
bin/rails generate migration CreateLoginEvents
```

O conteúdo está na seção **6. Associações**. Depois crie `app/models/login_event.rb` com o `belongs_to :user` e o `scope :recentes`, e acrescente no `User`:

```
has_many :login_events, dependent: :delete_all
```

```
bin/rails db:migrate
```

O que você deve ver, no console:

```
u = User.first || User.create!(name: "Ana", email: "ana@ufop.br", password: "Automic@2026",
  course: "Automação", matricula: "2011234", terms_accepted_at: Time.current)
u.login_events.create!(client: "mobile", occurred_at: Time.current)
u.login_events.count                # 1
u.login_events.recentes.first.user.name  # navega para o outro lado
```

Avançado: apague o usuário e confirme que o histórico foi junto.

```
LoginEvent.count    # 1
u.destroy
LoginEvent.count    # 0  <- é o dependent: :delete_all
```

Se der errado

Erro	Causa	Saída
<code>PG::ForeignKeyViolation</code>	você criou um <code>login_event</code> com <code>user_id</code> que não existe	crie sempre a partir do usuário: <code>u.login_events.create!</code>
<code>NameError: uninitialized constant User::LoginEvent</code>	o model não existe ou está no caminho errado	<code>app/models/login_event.rb</code>
<code>ActiveRecord::RecordInvalid: Validation failed: User must exist</code>	o <code>belongs_to</code> é obrigatório por padrão no Rails 5+	é isso mesmo; passe o usuário
<code>undefined method 'recentes'</code>	o scope não foi declarado	<code>scope :recentes, -> { order(oc</code>
apagou o usuário e sobraram os eventos	falta o <code>dependent:</code>	acrescente no <code>has_many</code>
<code>unknown attribute 'occurred_at'</code>	a migration não tem a coluna	confira o <code>schema.rb</code>

Prática 7: Os dois concerns

O momento em que os módulos da Aula 1 deixam de ser teoria.

Crie `app/models/concerns/email_verifiable.rb` e `app/models/concerns/password_resetable.rb`. O primeiro está inteiro na seção **7. Concerns**; o segundo é o mesmo desenho, trocando `email_verification_*` por `password_reset_*` e sem o `email_verified_at`.

Métodos que cada um precisa ter:

EmailVerifiable	PasswordResettable
email_verified?	—
issue_email_verification_code!	issue_password_reset_code!
verify_email_code!	verify_password_reset_code!
verification_expired?	password_reset_expired?
resend_verification_allowed?	password_reset_request_allowed?
—	clear_password_reset_code!

O que você deve ver, no console:

```
u = User.first
codigo = u.issue_email_verification_code!
codigo          # 6 dígitos
u.email_verification_code_digest # o hash, NUNCA o código
u.verify_email_code!(codigo)    # true
u.verify_email_code!(codigo)    # false – só vale uma vez
u.email_verified?              # true
```

Repare: o banco guarda o **digest** do código, não o código. Mesma lição da senha.

Se der errado

Erro	Causa	Saída
undefined method 'issue_email_verification_code!'	faltou o <code>include EmailVerifiable</code> no <code>User</code>	acrescente
NameError: uninitialized constant EmailVerifiable	o arquivo está fora de <code>app/models/concerns/</code>	mová
undefined method 'extend' / 'included'	faltou <code>extend ActiveSupport::Concern</code> no topo do módulo	acrescente
<code>verify_email_code!</code> devolve <code>true</code> duas vezes	o método não limpa o digest depois de usar	é a segunda linha do <code>verify_</code> código usado
<code>verify_email_code!</code> devolve <code>false</code> com o código certo	você comparou o código com o digest	compare com <code>BCrypt::Password.new(digest)</code>
o código expira na hora	<code>expires_at</code> está no passado	<code>15.minutes.from_now</code> , não

Prática 8: Fixtures, testes e commit

Fecha o dia, e é o que a Aula 3 vai usar de base.

Crie `test/fixtures/users.yml` com dois usuários: um verificado, outro não:

```
ana:
  name: Ana Souza
  email: ana@aluno.ufop.edu.br
  password_digest: <%= BCrypt::Password.create("Automic@2026") %>
  course: Engenharia de Controle e Automação
  matricula: "2011234"
  terms_accepted_at: <%= 1.day.ago.to_fs(:db) %>
  email_verified_at: <%= 1.day.ago.to_fs(:db) %>

bruno:
  name: Bruno Lima
  email: bruno@aluno.ufop.edu.br
  password_digest: <%= BCrypt::Password.create("Automic@2026") %>
  course: Engenharia de Minas
  matricula: "2019876"
  terms_accepted_at: <%= 1.day.ago.to_fs(:db) %>
  email_verified_at:
```

Depois escreva os testes de `user_test.rb`, dos dois concerns, do validador e do `login_event_test.rb`. Comece pelo que está na seção **10. Testando o model**.

```
bin/rails test
bin/rubocop
git add -A && git commit -m "Model User, concerns e associações" && git push
```

O que você deve ver: 30 testes verdes. O gabarito tem exatamente esse número.

Se der errado

Erro	Causa	Saída
<code>ActiveRecord::Fixture::FixtureError: table "users" has no column named X</code>	a fixture tem um campo que a tabela não tem	compare com o <code>schema.r</code>
<code>ActiveRecord::RecordNotUnique</code> nas fixtures	dois usuários com o mesmo e-mail ou matrícula	troque um deles
<code>NoMethodError: undefined method 'to_fs'</code>	Rails antigo	este material é Rails 8; confira <code>bin/rails -v</code>
um teste passa sozinho e falha junto com os outros	vazamento de estado entre testes	não use <code>User.first</code> no teste: use <code>users(:ana)</code>
<code>PendingMigrationError</code> só no teste	banco de teste atrasado	<code>bin/rails db:test:prepa</code>
<code>bin/rubocop</code> reclama de dezenas de coisas	estilo	<code>bin/rubocop -a</code> conserta maioria sozinho; leia o que sobrar
menos de 30 testes	falta cobrir algum caso	veja quais arquivos o gabarito tem em <code>test/</code>

Bônus, se sobrou tempo

1. Veja o N+1 acontecer. Com o log do Rails aberto:

```
User.all.each { |u| puts u.login_events.count } # uma consulta por usuário
User.includes(:login_events).each { |u| puts u.login_events.count } # duas, no total
```

2. Tente furar o índice único. Crie dois usuários com o mesmo e-mail direto no banco, sem passar pelas validações, e veja o Postgres recusar:

```
User.new(email: User.first.email, ...).save!(validate: false)
```

O erro que vem é `PG::UniqueViolation`, e é por isso que a restrição vive no banco, e não só no model.

Travou em qualquer prática? Compare arquivo por arquivo com o gabarito:

```
cd ~/capacitacao-gabarito && git checkout aula-02
cat app/models/user.rb
```

Recapitulando

- Migration é a fonte da verdade do banco. O model não declara colunas.
- Índice único é garantia; validação é mensagem bonita. Você quer os dois.
- Senha vira hash bcrypt, com salt, e nunca volta.
- Concern é o mixin do Rails: um assunto por arquivo.
- Normalize antes de validar, ou o índice único não serve para nada.
- A chave estrangeira fica no lado "muitos": `belongs_to` carrega, `has_many` é apontado.
- Transação é tudo ou nada. Dentro dela, use os métodos com `!`.
- `includes` resolve N+1, e N+1 é o problema de performance mais comum em Rails.

Na próxima: as rotas. O usuário vai conseguir se cadastrar, entrar, sair e recuperar a senha.